

9. Bölüm

Göstericiler

9.1 Gösterici nedir? Niçin İhtiyaç Duyarız?

Şu ana kadar gördüğümüz `char`, `int`, `long`, `float`, `double` tipli değişkenler ve bunlara ait diziler, değer tipler (**value type**) olarak adlandırılır. Çünkü kimliklendirilen değişken, değeri doğrudan ifade eder. Değer tiplerin yanı sıra değişkenlerin adreslerini tutan değişkenlere de ihtiyaç duyarız. İşte adres tutan bu değişkenlere **gösterici tipler** (**pointer type**) adını veririz. Göstericileri işaret ettikleri veri tipine (**data type**) göre tanımlarız. Bu nedenle göstericiler, **türetilmiş tipler** (**derived type**) olarak adlandırılır.

Göstericilerin kendisi ve İşaret Ettiği Veriler

Gösterici değişkenin kendisi yığın (**stack**) bellekte tutulur. İşaret ettiği veriler her bellekte olabilir ama genellikle öbek (**heap**) bellekteki verileri gösterir. Bu nedenle dikkatli kullanılmalıdır. C# ve Java gibi saf nesne yönelimli dillerde öbek bellekte çalışma anında oluşturulan nesneleri referans veri tipleri gösterir ve gösterici kullanımına kontrollü olarak izin verilir.

Gösterici tipler, gösterdiği yerdeki verileri değiştirmek için de kullanılır. **Gösterici tipler işaret ettiği verinin tipine göre tanımlanırlar. Bütün gösterici tipler bellekte aynı miktarda yer kaplar. Çünkü hepsi adres tutar.** Bütün bu tanımlamaların ışığında göstericileri, vatandaşların ikametlerini gösteren adres numaraları olarak düşünebiliriz.

Gösterici tiplere ihtiyaç duyulmasının sebebi hız ve esnekliktir. Genel olarak birkaç madde ile sıralayacak olursak;

- ◆ Belleğin istenilen bölgesine erişim sağlamak için göstericiler kullanılır. Günümüz modern dillerinde **referans tipler** (**referans type**) kullanılır. Göstericilerden farklı olarak referans tipler, gösterilen adreste bir verinin/nesnenin bulunduğunu garanti eder. Göstericiler her bellek bölgesine erişebilir.
- ◆ Bazen çalıştırma anında yeni bellek bölgelerine ihtiyaç duyarız bu durumda göstericiler gereklidir. **Dinamik veri yapıları** (**dynamic data structure**) göstericiler yardımıyla oluşturulup yönetilir.
- ◆ Fonksiyonlara parametre geçirilirken argüman olarak kullanılan yerel değişkenlerin kopyaları kullanılır. Bu durum Şekil 7.3'de açıklanmıştı. Fonksiyondan geri döndüğünde ise yerel değişkenler eski haline yığın bellekten geri çekilerek çevrilir. Fonksiyonun yerel değişken ya da argümanlardan birini değiştirmesi istendiğinde fonksiyona argüman olarak değişkenin adresi verilir. Böylece fonksiyona parametre olarak geçirilen yerel değişkenler göstericiler üzerinden fonksiyon içinde değiştirilebilir hale gelir.

Göstericilere olan ihtiyacı bir başka örnekle de açıklayabiliriz; Bir ormandaki ağaçları boylarına göre sıralamaya kalkarsak her birini yer değiştirme yöntemiyle sıralamak oldukça külfetli ve zaman alan bir işlemdir. Halbuki ağaçlara numara vererek ve bu numaraların karşısına uzunluklarını yazacağımız bir liste oluşturduğunda sadece numaraları sıralamak oldukça kolay ve zahmetsiz bir işlemdir. İşte buradaki numaralara karşılık gelen değişkenler göstericilerdir.

9.2 Gösterici Tanımlama

Göstericiler, işaret ettiği verinin tipine göre tanımlanırlar. Gösterici tanımlanırken veri tipi izleyen **başvuru kaldırma işleci** (**dereference operator**) olan yıldız kullanılır. Başvuru kaldırma deyimi, gösterici tarafından tutulan bellek adresinde saklanan değere erişmeyi ifade eder.

```
//GÖSTERİCİ TANIMLAMA:
veritipi *gostericiKimligi1; // float *ptrToFloat1;
veritipi* gostericiKimligi2; // float* ptrToFloat2;
```

```
//AYNI VERİ TİPİNDE TEK SATIRDA GÖSTERİCİ ve DEĞİŞKEN TANIMLAMA:
veritipi* gostericiKimligi3, degiskenKimligi1; // float* ptrToFloat3, agirlik;
veritipi degiskenKimligi2, *gostericiKimligi4; // float olcek, katsayi, *ptrToFloat5;
```

```
//KARIŞIKLIK OLMAMASI İÇİN EN DOĞRU KULLANIM:
veritipi *gosterici1, *gosterici2, degisken1,degisken2;
```

Göstericinin işaret ettiği değişkene değer atama aşağıdaki şekilde yapılır. Burada göstericinin veri tipi ile atanan değer aynı veri tipinde olmalıdır;

```
gostericikimligi = &degiskenkimligi;
```

Bir değişkenin adresinin **adres işleci** ile elde edilebileceği 4.7.2 başlığında anlatılmıştı. Aşağıda göstericilere ilişkin kod örnekleri verilmiştir;

```
char *ptrToChar; /*göstericinin işaret ettiği adreste "char" tipinde veri olan "ptrToChar"
→ kimlikli göstericisi tanımlandı.*/

int i=10;
int *ptrToInt=&i; /*göstericinin işaret ettiği adreste "int" tipinde veri olan "ptrToInt"
→ kimlikli göstericisi tanımlandı. Ancak göstericinin işaret ettiği adres olarak i değişkeninin
→ adresi atanıyor.*/

*ptrToInt =20; /* "ptrToInt" göstericisinin işaret ettiği adresteki tamsayı değeri 20 yaptık.
→ "ptrToInt" göstericisi, i değişkeninin adresini işaret ettiği i değişkeninin değeri de 20
→ olmuştur */
```

Jenerik Gösterici

Veri tiplerinin işlendiği 4.3 başlığında, **hiçbir değeri olmayan ve bellekte yer kaplamayan void** veri tipi anlatılmıştı. Veri tipi **void** olan **void*** göstericisine **jenerik gösterici** (**generic pointer**) adı verilir.

Bazı durumlarda göstericilerin veya gösterdiği değerlerin sabit olması gerekebilir. Bunun için üç durum ortaya çıkabilir;

1. **İşaret edilen değerın sabit olması durumu:** Bu durumda gösterici tanımlaması aşağıdaki gibi yapılır;

```
int main() {
    int i=10;
    const int *ptrToi=&i; // değerin sabit olması durumunda gösterici tanımlama
    *ptrToi=20; /* HATA: error: assignment of read-only location ‘*ptrToi’: pi
    → göstericisinin işaret ettiği adresteki tamsayı değeri 20 yapılamaz. */
    return 0;
}
```

2. **Göstericinin işaret ettiği adresin sabit olması durumu:** Kısaca göstericinin sabit olması durumunda tanımlama aşağıdaki gibi yapılır;

```
int main() {
    int i=10;
    int *const ptrToi=&i; // göstericinin sabit olması durumunda gösterici tanımlama
    /* Bu tür tanımlamada bir adres ilk değer (initialize) olarak mutlaka verilmelidir. */
    int j=30;
    *ptrToi=20; // pi göstericisinin işaret ettiği adresteki tamsayı değeri 20 olur.
    ptrToi =&j; /*HATA:error: assignment of read-only variable ‘ptrToi’
    pi göstericisinin işaret ettiği adres değiştirilemez!*/
    return 0;
}
```

3. **Hem göstericinin hem de değerin sabit olması durumu:**

```
int main() {
    int j=10, k=100;
    const int * const ptr=&j; /* Hem gösterici, hem de gösterilen veri sabit */
    /* Bu tür tanımlamada da bir adres ilk değer (initialize) olarak mutlaka verilmelidir.
    → */
    *ptr=20; /* HATA: error: assignment of read-only location ‘*(const int *)ptr’*/
    ptr=&k; /*HATA: error: assignment of read-only variable ‘ptr’ */
}
```

```
    return 0;
}
```

Gösterici tipe atamaya uygun kesin bir adresiniz yoksa, **NULL** değeri atamak her zaman iyi bir uygulamadır. İlk değeri olmayan gösterici tanımlamak yerine, ilk değeri **sıfır** olan göstericiler tanımlanmak doğru bir yöntemdir. Bu durumda gösterici, **boş gösterici (NULL Pointer)** adı verilir. Aşağıdaki gibi kullanılır;

```
veritipi *gostericikimligi = NULL;
gostericikimligi = NULL;
```

Aşağıda boş gösterici kullanımına ilişkin örnek bir uygulama verilmiştir;

```
#include <stdio.h>
int main(){
    int *ptr = NULL;
    printf("ptr göstericisinin işaret ettiği adres: %p\n", ptr);
    int agirlik=80;
    ptr=&agirlik;
    printf("ptr göstericisinin işaret ettiği adres: %p ve değeri: %d\n", ptr, *ptr);
    return 0;
}
/* Program Çıktısı:
ptr göstericisinin işaret ettiği adres: (nil)
ptr göstericisinin işaret ettiği adres: 0x7ffdedc33dec ve değeri: 80
*/
```

Bir göstericinin işaret ettiği yerde bir başka gösterici olabilir. Buna **çifte gösterici (double pointer)** adı verilir. Aşağıda buna ilişkin bir örnek verilmiştir;

```
#include <stdio.h>
int main(){
    int var = 10;
    int *intPtr = &var;
    int **ptrToPtr = &intPtr; //double pointer
    printf("var:%d &var:%p \n",var, &var);
    printf("inttPtr: %p &inttPtr:: %p \n\n", intPtr, &intPtr);
    printf("var: %d *intPtr: %d \n", var, *intPtr);
    printf("ptrToPtr: %p &ptrToPtr: %p \n", ptrToPtr, &ptrToPtr);
    printf("&intPtr: %p *ptrToPtr : %p \n\n", &intPtr, *ptrToPtr);
    printf("var: %d *intPtr: %d **ptrToPtr: %d", var, *intPtr, **ptrToPtr);
    return 0;
}
/*Program Çıktısı:
var:10 &var:0x7fff67f1b2bc
inttPtr: 0x7fff67f1b2bc *inttPtr: 0x7fff67f1b2b0

var: 10 *intPtr: 10
ptrToPtr: 0x7fff67f1b2b0 &ptrToPtr: 0x7fff67f1b2a8
&intPtr: 0x7fff67f1b2b0 *ptrToPtr : 0x7fff67f1b2bc

var: 10 *intPtr: 10 **ptrToPtr: 10
*/
```

Bir tam sayı bellekte 4 bayt yer işgal eder. Bu baytların bellekte nasıl yerleştiğini gösteren program verilmiştir.

```
#include <stdio.h>
int main() {
    int i=0x10203040;
    char *p= (char*) &i;
    /* int işaret eden adres, char içeriyor diye (char*) ifadesiyle tip dönüşümü (cast)
    → yapılıyor. */
    printf("Sayı :%10d-%x\n",i,i);
    printf("1.bayt:%10d-%8x\n",*p,*p);
    printf("2.bayt:%10d-%6x\n",*(p+1),*(p+1));
    printf("3.bayt:%10d-%4x\n",*(p+2),*(p+2));
}
```

```

    printf("4.bayt:%10d-%x\n",*(p+3),*(p+3));
    return 0;
}
/*Program Çıktısı:
Sayı : 270544960-10203040
1.bayt:      64-      40
2.bayt:      48-      30
3.bayt:      32-      20
4.bayt:      16-      10
*/

```

9.3 Gösterici Aritmetiği

Göstericilerde toplama ve çıkarma tam sayılardan farklı şekilde çalışır. **Bir gösterici artırıldığında veya azaltıldığında, işaret ettiği adres, işaret ettiği veri tipinin bellek boyutu kadar artırılır veya azaltılır.** Bunun nedeni bilgisayarların tasarımından ötürü işlemciye bağlı belleklerde her bir bayt ayrı adreslenmesidir.

Bir `char*` göstericisi, işaret ettiği yerdeki `char` verisinden bir sonraki `char` verisini göstermesi istendiğinde **gösterici adresi 1 artırır**. Çünkü `char` veri tipi bellekte 1 bayt yer kaplar. Aşağıdaki program çıktısındaki adres değişimlerini inceleyiniz;

```

#include <stdio.h>
#define BOYUT 5
void printDizi(char *d) {
    printf("DİZİ:{");
    for (int i=0;i<BOYUT;i++)
        printf("\'%c\\',", d[i]);
    printf("}\n");
}
int main() {
    char dizi[BOYUT]={'I','L','H','A','N'}; /* Bellekte art arda 5 char'dan oluşan dizi. */

    char *ptrToChar=&dizi[0]; /* ptrToChar göstericisi dizinin ilk elemanını işaret ediyor.*/
    printf("İşaret Edilen Adres: %p\n",ptrToChar);
    *ptrToChar='X'; /* Göstericinin işaret ettiği yerdeki char değeri 'X' olarak değiştirildi. */
    printDizi(dizi);

    ptrToChar++; /* ptrToChar göstericisi bir sonraki baytı gösterecek şekilde artırıldı. */
    printf("İşaret Edilen Adres: %p\n",ptrToChar);
    *ptrToChar='Y'; /* Göstericinin işaret ettiği yerdeki char değeri 'X' olarak değiştirildi.*/
    printDizi(dizi);

    ptrToChar+=3; /* ptrToChar göstericisi üç sonraki baytı gösterecek şekilde artırıldı. */
    printf("İşaret Edilen Adres: %p\n",ptrToChar);
    *ptrToChar='Z'; /* Göstericinin işaret ettiği yerdeki char değeri 'X' olarak değiştirildi.*/
    printDizi(dizi);
    return 0;
}
/*Program Çıktısı:
İşaret Edilen Adres: 0x7ffd59be4313
DİZİ:{'X','L','H','A','N',}
İşaret Edilen Adres: 0x7ffd59be4314
DİZİ:{'X','Y','H','A','N',}
İşaret Edilen Adres: 0x7ffd59be4317
DİZİ:{'X','Y','H','A','Z',}
*/

```

Benzer şekilde `int*` göstericisi, işaret ettiği yerdeki `int` verisinden bir sonraki `int` verisini göstermesi istendiğinde **gösterici adresi 4 artırır**. Çünkü `int` veri tipi bellekte 4 bayt yer kaplar. Aşağıdaki program çıktısındaki adres değişimlerini inceleyiniz;

```

#include <stdio.h>
#define BOYUT 5
void printDizi(int *d) {

```

```

printf("DİZİ:{");
    for (int i=0;i<BOYUT;i++)
        printf("%d,", d[i]);
    printf("}\n");
}
int main() {
    int dizi[BOYUT]={10,20,30,40,50}; /* Bellekte art arda 5 int'den oluşan dizi. */

    int *ptrToint=&dizi[0]; /* ptrToint göstericisi dizinin ilk elemanını işaret ediyor.*/
    printf("İşaret Edilen Adres: %p\n",ptrToint);
    *ptrToint=-1; /* Göstericinin işaret ettiği yerdeki int değeri -1 olarak değiştirildi.*/
    printDizi(dizi);

    ptrToint++; /* ptrToint göstericisi bir sonraki int'i gösterecek şekilde artırıldı.*/
    printf("İşaret Edilen Adres: %p\n",ptrToint);
    *ptrToint=-2; /* Göstericinin işaret ettiği yerdeki int değeri -2 olarak değiştirildi.*/
    printDizi(dizi);

    ptrToint+=3; /* ptrToint göstericisi üç sonraki int'i gösterecek şekilde artırıldı.*/
    printf("İşaret Edilen Adres: %p\n",ptrToint);
    *ptrToint=-3; /* Göstericinin işaret ettiği yerdeki int değeri -3 olarak değiştirildi.*/
    printDizi(dizi);
    return 0;
}
/*Program Çıktısı:
İşaret Edilen Adres: 0x7ffec841be70
DİZİ:{-1,20,30,40,50,}
İşaret Edilen Adres: 0x7ffec841be74
DİZİ:{-1,-2,30,40,50,}
İşaret Edilen Adres: 0x7ffec841be80
DİZİ:{-1,-2,30,40,-3,}
*/

```

Bunun gibi;

- ◊ **short*** göstericisi, işaret ettiği yerdeki **short** verisinden bir sonraki **short** verisini göstermesi istendiğinde gösterici adresi 2 artar. Çünkü **short** veri tipi bellekte 2 bayt yer kaplar.
- ◊ **long*** göstericisi, işaret ettiği yerdeki **long** verisinden bir sonraki **long** verisini göstermesi istendiğinde gösterici adresi 8 artar. Çünkü **long** veri tipi bellekte 8 bayt yer kaplar;
- ◊ **float*** göstericisi, işaret ettiği yerdeki **float** verisinden bir sonraki **float** verisini göstermesi istendiğinde gösterici adresi 4 artar. Çünkü **float** veri tipi bellekte 4 bayt yer kaplar.
- ◊ **double*** göstericisi, işaret ettiği yerdeki **double** verisinden bir sonraki **double** verisini göstermesi istendiğinde gösterici adresi 8 artar. Çünkü **double** veri tipi bellekte 8 bayt yer kaplar;

Ayrıca göstericinin artırılması yanında eksiltilmesi de mümkündür. Bu durumda -, -- ve ++ işlemleri kullanılır.

Bilgi

- ◊ Görüldüğü üzere göstericilerin üzerinde işlemler (**operator**) işlem yaparken, göstericinin işaret ettiği verinin tipine göre farklı işlem yaparlar.
- ◊ Gösterici aritmetikinde (++ , -- , + , - , += ve -=) ile (< , > , >= , <= , != ve ==) çalışır. Diğer işlemler çalışmaz.

9.4 Gösterici Dizileri

Göstericiler de dizi olarak tanımlanabilir. Aşağıda birbirinden bağımsız tanımlanmış yerel değişken adreslerinden bir gösterici dizisi tanımlanmıştır. Bu dizi üzerinden değişkenler daha kolay işlenebilir;

```

#include <stdio.h>
int main() {
    int i = 10, j=20;
    float f=1.0;
    int k=30, l=40;

```

```

int *ptr[4]; /* int gösteren 4 gösteri içeren bir dizi tanımlandı */
/* Göstericilere ilk değer veriliyor*/
ptr[0] = &i; // Birinci eleman i değişkeninin adresini
ptr[1] = &j; // İkinci eleman j değişkeninin adresini
ptr[2] = &l; // Üçüncü eleman l değişkeninin adresini
ptr[3] = &k; // Dördüncü eleman k değişkeninin adresini
*ptr[2] = -1; // Üçüncü elemanın işaret ettiği adresteki değer -1 olarak değiştirildi. (l
→ değişkeni)
// Değerlere Ulaşma
int indis;
for (indis = 0; indis < 4; indis++)
    printf("ptr[%d] ile gösterilen değer: %d\n", indis, *ptr[indis]);

return 0;
}
/*Program Çıktısı:
ptr[0] ile gösterilen değer: 10
ptr[1] ile gösterilen değer: 20
ptr[2] ile gösterilen değer: -1
ptr[3] ile gösterilen değer: 30
*/

```

9.5 Referansla Çağırma

Fonksiyonlara parametre geçirilirken argüman olarak kullanılan yerel değişkenlerin kopyaları kullanılır. Bu durum Şekil 7.3'de açıklanmıştı. Fonksiyondan geri döndüğünde ise yerel değişkenler eski haline yığın bellekten geri çekilerek çevrilir. Buna ilişkin örnek aşağıda verilmiştir;

```

#include <stdio.h>
void degistir(int,int);
int main(){
    int a = 10, b = 20;
    degistir(a, b);
    // çağrı sonrası "yerel değişkenler" yığın bellekten geri çekilerek eski haline gelmiştir.
    printf("1- a:%d, b:%d\n", a, b); //Çıktı: a:10, b:20
}
void degistir(int pX, int pY) {
    int z = pX;
    pX=pY;
    pY=z;
}

```

Fonksiyona geçirilen yerel değişken ya da argümanlardan birinin değiştirilmesi istendiğinde bu fonksiyona argüman olarak değişkenin adresi verilir. Böylece fonksiyona parametre olarak geçirilen yerel değişkenler dolaylı olarak göstericiler üzerinden fonksiyon içinde değiştirilebilir hale gelir. Fonksiyona geçirilecek argümanlar gösterici olarak tanımlanır. Çağrı ortamına geri döndüğünde yerel değişkenler de değişmiş olur. Bu işleme **referansla çağırma** (**call by reference**) adı verilir. Çünkü fonksiyona değişken adresleri değer olarak geçirilmiştir. Buna ilişkin örnek aşağıda verilmiştir;

```

#include <stdio.h>
void degistir(int *pX, int *pY);
int main(){

    int a = 10, b = 20;
    degistir(&a, &b);
    // çağrı sonrası "yerel değişkenlerin adresleri" yığın bellekten geri çekilerek eski haline
    → gelmiştir.
    printf("2- a:%d, b:%d\n", a, b); //Çıktı: a:20, b:10
}
void degistir(int* pX, int* pY){
    int z = *pX;
    *pX=*pY;
    *pY=z;
}

```

```
}
```

9.6 Göstericilerin Dizileri İşaret Etmesi

Çoğu durumda, bir dizi yardımıyla gerçekleştirdiğimiz işleri gösterici ile de gerçekleştirilebiliriz. Dizi değişkeni, dizinin ilk öğesinin adresi işaret eder. Kısaca dizi değişkenleri gösterici gibi davranırlar.

```
#include <stdio.h>
int main () {
    int dizi[5] = {10, 20, 30, 40, 50};
    int *ptr = dizi; // int *ptr=& dizi[0]; olarak da kodlanabilir.
    int indis;
    printf("Dizi elemanlarına gösterici ile erişim: \n");
    for(i = 0; i < 5; i++) {
        printf("dizi[%d]: %d\n", indis, *(ptr+indis));
    }
    return 0;
}
```

Dizi Değişkenleri

Fonksiyonlara dizileri işaret eden göstericileri parametre olarak verebiliriz. Dizi değişkeni, dizinin ilk öğesinin adresi işaret ettiğinden dolayı **dizi değişkenleri (array)** parametre olarak kullanılırken adres işleci (**address operator**) **kullanılmaz!** Bu durumda parametre olarak dizi elemanının veri tipinde bir göstericiyi parametre olarak tanımlarız.

```
float notlar[10];
float notlarOrtalamasi(float*); //prototype
//...
float ort=notlarOrtalamasi(notlar); // yada notlarOrtalamasi(&notlar[0]);
//...
float matris[4][3];
float defetminant(float**,int,int); //prototype
//...
float det=defetminant(matris,4,3);
//...
```

Aşağıda öğrencilerin notlarına ilişkin işlemler yapılan bir program verilmiştir.

```
#include <stdio.h>
#include <math.h>
#define OGRENCISAYISI 10
float notlarOrtalamasi(float*); // parametre olarak dizi kabul eder
void notlariArtir(float*); // parametre olarak dizi kabul eder
int main(){
    float notlar[OGRENCISAYISI]= { 55.0,60.0,70.0,35.0,30.0,50.0,65.0,90.0,95.0,100.0 };
    float toplam=notlarOrtalamasi(notlar);
    printf("Notlar Ortalaması: %f\n", toplam);
    notlariArtir(notlar); // Bu fonksiyonla dizi elemanları değiştiriliyor
    int indis;
    for (indis=0; indis<OGRENCISAYISI; indis++)
        printf("Not[%d]=%f\n",indis,notlar[indis]);
    return 0;
}
float notlarOrtalamasi(float* pNotlar){
    int indis;
    float top=0.0;
    for (indis=0; indis<OGRENCISAYISI; indis++)
        top+=pNotlar[indis]; // Gösterici dizi gibi kullanılıyor
    return top/OGRENCISAYISI;
}
void notlariArtir(float* pNotlar) {
    int indis;
```



```

    for (indis=0; indis<OGRENCISAYISI; indis++)
        if (pNotlar[indis]<=90) //Göstericinin gösterdiği değerler değiştiriliyor
            pNotlar[indis]+=10.0;
};
/*Program Çıktısı:
Notlar Ortalaması: 65.000000
Not[0]=65.000000
Not[1]=70.000000
Not[2]=80.000000
Not[3]=45.000000
Not[4]=40.000000
Not[5]=60.000000
Not[6]=75.000000
Not[7]=100.000000
Not[8]=95.000000
Not[9]=100.000000
*/

```

9.7 Fonksiyonlardan Bir Diziyi Geri Döndürme

C dilinde bir fonksiyonun geri dönüş değeri olarak tüm bir dizi verilemez! Ancak, bir dizinin göstericisini fonksiyondan geri dönüş değeri olarak döndürebilirsiniz. **Burada dikkat edilmesi gereken fonksiyondan çıkılınca dizinin bellekten kaldırılmamasıdır.** Çünkü yerel değişkenler yığın bellekte tutulduğundan fonksiyondan çıkılınca bellekten silinirler. Aşağıda bir tamsayı dizi göstericisini döndüren bir fonksiyon örnekleri verilmiştir;

```

int* tekBoyutluDiziDondur() {
    /*... */
}
int** ikiBoyutluDiziDondur() {
    /*... */
}

```

Yerel değişkenlerin yığın bellekte (**stack segment**) tutulduğu ve fonksiyondan geri dönünce fonksiyon yerelindeki değişkenlerin kaybolduğu 7.5.1 başlığında ve Şekil 7.3’de anlatılmıştı. Bu nedenle bir fonksiyon içinde tanımlanan bir dizinin göstericisi geri döndürülecek ise veri bellekte (**data segment**) yer almasını sağlamak için **static** olarak tanımlanmak zorundadır.

```

#include <stdio.h> // printf() ve scanf()
#include <stdlib.h> // rand() ve srand()
#include <time.h> // time()
#define BOYUT 10
int* rastgeleOgrenciNotlari( ) {
    static int notlar[BOYUT]; // Fonksiyondan çıkıldığında bellekten kaldırılmaması için
    ↪ "static" olarak tanımlanmıştır!

    for (int i = 0; i < BOYUT; ++i)
        notlar[i] = rand()%101;
    return notlar;
}
void notlariYaz(int* pNotlar,int pUzunluk) {
    printf("Öğrenci Notları:\n");
    for (int i = 0; i < pUzunluk; ++i)
        printf("%4d", pNotlar[i]);
    return;
}
int main () {
    int *notlar;
    srand(time(NULL));
    notlar = rastgeleOgrenciNotlari();
    notlariYaz(notlar,BOYUT);
    return 0;
}

```



```

}
/*Program Çıktısı:
Öğrenci Notları:
30 31 13 40 41 37 34 83 93 76
*/

```

9.8 Fonksiyon Göstericisi

Geri çağırma (callback) fonksiyonları, özellikle **olay güdümlü programlamada (event-driven programming)** çok kullanışlıdır. Bir olay tetiklendiğinde, bu olaylara yanıt olarak ona eşlenen bir geri çağırma fonksiyonu çalıştırılır. Genellikle grafik ara yüzülü işletim sistemlerinde kullanıcı ara yüzünde bir düğmeye tıklanması gibi bir olay, önceden tanımlanmış bir dizi işlemi başlatabilir. Bu durum, geri çağırma işlevleriyle gerçekleştirilir.

Geri çağırma fonksiyonu, temel olarak, başka bir koda argüman olarak geçirilen ve belirli bir zamanda argümanı geri çağırması beklenen bir icra edilebilir koddur. Geri çağırma mekanizması fonksiyon göstericisine bağlıdır. **Fonksiyon göstericisi (function pointer)**, bir fonksiyonun bellek adresini depolayan bir değişkendir. Aşağıda buna ilişkin basit bir örnek bulunmaktadır;

```

#include <stdio.h>
void merhaba() {
    printf("Merhaba!\n");
}
void callback(void (*ptr)()) {
    printf("Geri çağırma fonksiyonu çağrılacak!\n");
    (*ptr)(); // Geri çağırma fonksiyonu çağrılıyor
}
int main() {
    void (*ptr)() = merhaba;
    callback(ptr);
    return 0;
}

```

9.9 Düşün ve Çöz

- ◇ **Düşün:** "Bir gösterici aslında sadece bir tam sayıdır (adres)" ifadesi doğru mudur? Gösterici tipi (`int*`, `char*` vb.) gösterici aritmetiğini nasıl etkiler?
- ◇ **Düşün:** `const int *p;` ile `int * const p;` tanımları arasındaki farkı bellek ve erişim yetkisi açısından açıklayınız.
- ◇ **Düşün:** Göstericiler ile diziler arasındaki adres-offset ilişkisini, dizi ismiyle gösterici aritmetiği yaparak açıklayınız.
- ◇ **Çöz:** Bir fonksiyon yazınız; bu fonksiyon iki tam sayının adresini parametre olarak alsın ve bu sayıların değerlerini bellekte yer değiştirsin.
- ◇ **Çöz:** Bir metni (**string**) gösterici aritmetiği kullanarak tersten ekrana yazdıran bir fonksiyon yazınız.
- ◇ **Çöz:** Bir fonksiyona parametre olarak başka bir fonksiyonun adresini gönderen (**function pointer**) basit bir matematiksel işlem seçici (topla/çıkart) tasarlayın.
- ◇ **Çöz:** `&` (adres) ve `*` (içerik/indirection) operatörlerinin işlevlerini basit bir örnek değişken ve gösterici üzerinde açıklayınız.
- ◇ **Çöz:** 32-bit bir sistemde `char*`, `int*` ve `double*` gösterici değişkenlerinin bellekte kapladığı alanlar kaç bayttır? Neden?
- ◇ **Çöz:** `int A[5] = {10, 20, 30, 40, 50};` ve `int *p = A;` tanımları veriliyor. `*(p + 2)` ve `*p + 2` ifadelerinin değerlerini ayrı ayrı hesaplayınız.
- ◇ **Çöz:** İki `int` değişkenin değerlerini birbiriyle takas eden (**swap**) fonksiyonu göstericiler kullanarak yazınız.
- ◇ **Çöz:** C dilinde dizi adının aslında dizinin ilk elemanının adresini tutan sabit bir gösterici olduğunu kanıtlayan bir örnek kod yazınız.
- ◇ **Çöz:** `int *ptr_arr[3];` şeklinde tanımlanan "gösterici dizisi" ile `int (*arr_ptr)[3];` şeklinde tanımlanan "dizi göstericisi" arasındaki farkı bellek şeması çizerek açıklayınız.
- ◇ **Çöz:** İki `int` parametre alan ve `int` döndüren bir fonksiyonu işaret eden fonksiyon göstericisi tanımlayınız. Bu göstericiyi topla(`int, int`) fonksiyonuna bağlayıp çağırınız.
- ◇ **Çöz:** Bir fonksiyon içerisindeki yerel (**auto**) bir dizinin adresini `return` ile döndürmek neden tehlikelidir ve Tanımsız Davranışa (**Undefined Behavior**) yol açar?
- ◇ **Çöz:** `double` türünde bir göstericiye 1 eklendiğinde (`p++`), bellek adres değeri sayısal olarak kaç artar?

Mantığını açıklayınız.

- ◇ **Çöz:** Aşağıdaki kod bloğu yürütüldüğünde ekrana ne basılır?

```
int nums[] = {100, 200, 300};  
int *ptr = nums;  
printf("%d, ", *ptr++);  
printf("%d, ", *ptr);  
printf("%d\n", (*ptr)++);
```